

SI1001 Teoría de la Computación

Frama-C y su plugin WP

Andrés Sicard Ramírez

Universidad EAFIT

Semestre 2025-2

Preliminares

Convenciones

- El texto guía para esta sección es (Blanchard 2024).
- La numeración (de secciones, definiciones, teoremas, figuras, páginas, etc.) en esta sección corresponde a la numeración en el texto guía.

Introducción

Descripción

- Frama-C (*FRAmework for Modular Analysis of C code*) es una plataforma para análisis de programas escritos en C creada por la CEA LIST y el Inria.
- ACSL (*ANSI/ISO C Specification Language*) (pronunciado «Axel») es el lenguaje de especificación para ANSI/ISO C usado por Frama-C.
- WP (*Weakest Precondition*) es un plugin para realizar verificación formal de programas en Frama-C.

Introducción

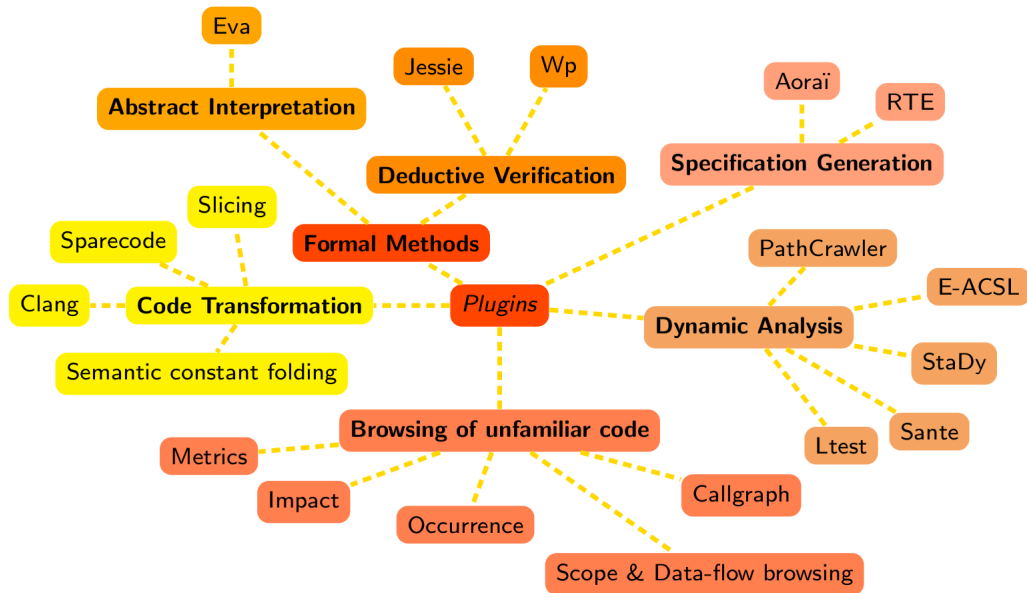


Figura (pág. 13).

Contratos para funciones

*«The goal of a **function contract** is to state the properties of the input that are expected by the function, and in exchange the properties that will be assured for the output.»* (pág. 22)

Los contratos para funciones están compuestos de **pre-condiciones** y **post-condiciones**, las cuales se escriben en ACSL.

Contratos para funciones

Ejemplo

Calcular el valor absoluto de un número entero ([src/frama-c/abs-1.c](#)).

```
1 int abs(int val) {  
2     if ( val < 0 ) return -val;  
3     return val;  
4 }
```

(continua en la próxima diapositiva)

Contratos para funciones

Ejemplo (continuación)

Adicionamos una primera post-condición escrita en ACSL (`src/frama-c/abs-2.c`).

```
1  /*@
2    ensures \result >= 0;
3  */
4  int abs(int val) {
5    if ( val < 0 ) return -val;
6    return val;
7  }
```

(continua en la próxima diapositiva)

Contratos para funciones

Ejemplo (continuación)

Adicionamos una segunda post-condición escrita en ACSL (`src/frama-c/abs-3.c`).

```
1  /*@
2    ensures \result >= 0;
3    ensures (val >= 0 ==> \result == val) &&
4              (val < 0 ==> \result == -val);
5  */
6  int abs(int val) {
7    if ( val < 0 ) return -val;
8    return val;
9  }
```

(continua en la próxima diapositiva)

Contratos para funciones

Ejemplo (continuación)

Ejecutamos las instrucciones y demostramos las post-condiciones utilizando el plugin WP.

```
$ cd src/frama-c  
$ frama-c-gui abs-3.c
```

Contratos para funciones

Pregunta

¿Es nuestro programa libre de errores? ¿Qué acerca de errores en tiempo de ejecución?

Contratos para funciones

El plugging RTE

El plugging RTE (*runtime errors*) adiciona anotaciones al programa para evitar errores en tiempo de ejecución (*integer overflow*, *invalid pointer dereferencing*, división por cero, etc.)

Contratos para funciones

Representación de números enteros

La anotación adicionada por el plugging RTE es debido a la representación de números enteros en el computador. Como es señalado por *The GNU C Library Reference Manual* para la versión 2.36:

Most computers use a two's complement integer representation, in which the absolute value of `INT_MIN` (the smallest possible `int`) cannot be represented; thus, `abs (INT_MIN)` is not defined.

Contratos para funciones

Representación de enteros con signo en complemento a dos

Un sistema de representación binaria en complemento a dos con n bits tiene las siguientes características:

- El sistema representa 2^n enteros con signo entre los valores -2^{n-1} y $2^{n-1} - 1$.
- El bit más significativo representa el signo del número entero: 0 representa un entero positivo y 1 representa un entero negativo.
- La representación del número entero $-x$ es el número binario que representa al valor $2^n - x$.
- Para representar el número entero $-x$, se escribe x en binario, se intercambian los ceros por unos y se adiciona uno al resultado.
- La representación del cero es única.

Contratos para funciones

Representación de enteros con signo en complemento a dos

Un sistema de representación binaria en complemento a dos con n bits tiene las siguientes características:^a

- El sistema representa 2^n enteros con signo entre los valores -2^{n-1} y $2^{n-1} - 1$.
- El bit más significativo representa el signo del número entero: 0 representa un entero positivo y 1 representa un entero negativo.
- La representación del número entero $-x$ es el número binario que representa al valor $2^n - x$.
- Para representar el número entero $-x$, se escribe x en binario, se intercambian los ceros por unos y se adiciona uno al resultado.
- La representación del cero es única.

^aVéase, por ejemplo, (Nisan y Shimon [2005] 2021).

Contratos para funciones

Ejemplo

Números enteros con signo representados en complemento a dos empleando 3-bits.

000: 0

001: 1

010: 2

011: 3

100: -4

101: -3

110: -2

111: -1

Contratos para funciones

Ejemplo

- Un `int` de C en GCC es representado con 32 bits (en mi computador), es decir, los valores que se pueden representar son

$$[-2^{31}, 2^{31} - 1] = [-2147483648, 2147483647].$$

- Un `Int` de Haskell en GHC es representado con 64 bits (en mi computador), es decir, los valores que se pueden representar son

$$[-2^{63}, 2^{63} - 1] = [-9223372036854775808, -9223372036854775807].$$

Contratos para funciones

Ejemplo

Programa que muestra el problema empleando la función `abs` de la *GNU C library* (`src/frama-c/src/glibc.c`).

```
1  #include <limits.h>
2  #include <stdio.h>
3  #include <stdlib.h>
4
5  int main() {
6      printf("Maximum int (INT_MAX, 2^31 - 1): %d\n", INT_MAX);
7      printf("Minimum int (INT_MIN, -2^31): %d\n", INT_MIN);
8      printf("Absolute value of INT_MIN (error): %d\n", abs(INT_MIN));
9  }
```

(continua en la próxima diapositiva)

Contratos para funciones

Ejemplo (continuación)

Compilar y ejecutar el programa con las siguientes instrucciones:

```
$ cd src
$ make glibc-abs
gcc-15.1.0 -Wall -Wextra -Wpedantic -Werror -std=c23 \
  -o glibc-abs glibc-abs.c
$ ./glibc-abs
Maximum int (INT_MAX, 2^31 - 1): 2147483647
Minimum int (INT_MIN, -2^31): -2147483648
Absolute value of INT_MIN (error): -2147483648
```

Contratos para funciones

Ejemplo

Programa que muestra el problema empleando la función `abs` de GHC ([src/frama-c/src/ghc-abs.hs](#)).

```
1  main :: IO ()
2  main = do
3      let maxInt = maxBound :: Int
4          minInt = minBound :: Int
5      print $ "Maximum Int (maxBound :: Int, 2^63 - 1): " ++ show maxInt
6      print $ "Minimum Int (minBound :: Int, -2^63): " ++ show minInt
7      print $ "Absolute value of (minBound :: Int) (error): " ++
8          show (abs minInt)
```

(continua en la próxima diapositiva)

Contratos para funciones

Ejemplo (continuación)

Compilar y ejecutar el programa con las siguientes instrucciones:

```
$ cd src
$ make ghc-abs
ghc -Wall -Werror -o ghs-abs ghc-abs.hs
$ ./ghc-abs
"Maximum Int (maxBound :: Int, 2^63 -1): 9223372036854775807"
"Minimum Int (minBound :: Int, -2^63): -9223372036854775808"
"Absolute value of (minBound :: Int) (error): -9223372036854775808"
```

Contratos para funciones

Ejemplo

Adicionamos la pre-condición asociada a la representación de los números enteros.
(`src/frama-c/abs-4.hs`).

```
1  #include <limits.h>
2
3  /*@
4     requires INT_MIN < val;
5     ensures \result >= 0;
6     ensures (val >= 0 ==> \result == val) &&
7             (val < 0 ==> \result == -val);
8  */
9  int abs(int val) {
10     if ( val < 0 ) return -val;
11     return val;
12 }
```

Contratos para funciones

Ejemplo (continuación)

Ejecutamos Frama-C con las siguientes instrucciones:

```
$ cd src/frama-c  
$ frama-c-gui -wp -wp-rte abs-4.c
```

Contratos para funciones

Ejemplo

También podemos verificar que las llamadas a la función `abs` satisfagan las pre-condiciones ([src/frama-c/abs-5.hs](#)).

```
1 void foo(int a) {  
2     int b = abs(42);  
3     int c = abs(-42);  
4     int d = abs(a);          // Warning: "a" may be INT_MIN  
5     int e = abs(INT_MIN);    // False: the parameter is INT_MIN  
6 }
```

Contratos para funciones

La función `main`

La función `main` es el punto de entrada del programa. Por esta razón al contexto de su evaluación Frama-C adiciona el valor inicial de la variables globales.

Contratos para funciones

Ejemplo

Una función `main` (`src/frama-c/main.c`).

```
1  int h = 42;
2
3  //@ requires 0 <= h <= 100 ;
4  int main() {
5      //@ assert h == 42 ;
6  }
7
8  //@ requires 0 <= h <= 100 ;
9  void foo() {
10     //@ assert h == 42 ;
11 }
```

(continua en la próxima diapositiva)

Contratos para funciones

Ejemplo (continuación)

Ejecutamos Frama-C con las instrucciones

```
$ cd src/frama-c  
$ frama-c-gui -wp -wp-rte main.c
```

Contratos para funciones

Apuntadores

El libro (Kernighan y Ritchie [1978] 1988) comienza la presentación de los apuntadores de la siguiente manera:

«A pointer is a variable that contains the address of a variable. Pointers are much used in C, partly because they are sometimes the only way to express a computation, and partly because they usually lead to more compact and efficient code than can be obtained in other ways...» (pág. 83)

*«Pointers have been lumped with the »**goto** statement as a marvelous way to create impossible-to-understand programs. This is certainly true when they are used carelessly, and it is easy to create pointers that point somewhere unexpected. With discipline, however, pointers can also be used to achieve clarity and simplicity.» (pág. 83)*

Contratos para funciones

Apuntadores

En C:

- el operador unario `&` da la dirección de un objeto (variable, función, etc) y
- el operador unario `*` es el operador de desreferenciación «*dereferencing operator*», que cuando aplicado a un apuntador, accede al objeto apuntado por el apuntador.

Contratos para funciones

Ejemplo

```
1  int x = 1;
2  int y = 2;
3  int* p;  /* ip is a pointer to int */
4
5  p = &x;  /* p now points to x */
6  y = *p;  /* y is now 1 */
7  *p = 0;  /* x is now 0 */
```

Contratos para funciones

Paso de parámetros a una función

El paso de parámetros a una función puede ser **por valor** «*call by value*» o **por referencia** «*call by reference*».

- Paso por valor: La función recibe el valor de la variable que se pasa y crea una copia local de ésta. En consecuencia, no es posible modificar la variable pasada.
- Paso por referencia: La función recibe la dirección de la variable que se pasa. En consecuencia, los cambios realizados por la función o el método se realizarán sobre la variable pasada.

En C utilizamos apuntadores para pasar los argumentos por referencia.

Contratos para funciones

Ejemplo

Contrato para un función recibiendo los argumentos por referencia
(src/frama-c/swap-1.c).

```
1  /*@
2     ensures *a == \old(*b) && *b == \old(*a);
3  */
4  void swap(int* a, int* b) {
5      int tmp = *a;
6      *a = *b;
7      *b = tmp;
8  }
```

(continua en la próxima diapositiva)

Contratos para funciones

Ejemplo (continuación)

Debemos verificar que los apuntadores sean válidos (`src/frama-c/swap-2.c`).

```
1  /*@
2    requires \valid(a) && \valid(b);
3    ensures  *a == \old(*b) && *b == \old(*a);
4  */
5  void swap(int* a, int* b) {
6    int tmp = *a;
7    *a = *b;
8    *b = tmp;
9  }
```

(continua en la próxima diapositiva)

Contratos para funciones

Ejemplo (continuación)

¿Por qué falla el contrato? (`src/frama-c/swap-3.c`).

```
1  int h = 42;  
2  
3  int main() {  
4      int a = 37;  
5      int b = 91;  
6  
7      //@ assert h == 42;  
8      swap(&a, &b);  
9      //@ assert h == 42;  
10 }
```

(continua en la próxima diapositiva)

Contratos para funciones

Ejemplo (continuación)

Debemos especificar los efectos colaterales «*side effects*» de la función (`src/frama-c/swap-4.c`).

```
1  /*@
2    requires \valid(a) && \valid(b);
3    assigns *a, *b;
4    ensures  *a == \old(*b) && *b == \old(*a);
5  */
6  void swap(int* a, int* b) {
7      int tmp = *a;
8      *a = *b;
9      *b = tmp;
10 }
```

Referencias



Allan Blanchard (2 de nov. de 2024). Introduction to C program proof with Frama-C and its WP plugin. Recent version from the GitHub repository. URL: https://github.com/AllanBlanchard/tutoriel_wp (vid. pág. 2).



Brian W. Kernighan y Dennis M. Ritchie [1978] (1988). The C Programming Language. 2.^a ed. Software Series. Prentice-Hall (vid. pág. 27).



Noam Nisan y Schocken Shimon [2005] (2021). The Elements of Computing Systems. Building a Modern Computer from First Principles. 2.^a ed. MIT Press (vid. pág. 14).